

We saw how to make a logical replica in practice and have addressed some of the issues that can occur when we work with logical replication.

In the next chapter, we'll talk about useful tools and extensions. We will see which tools are best to make life easier for a PostgreSQL DBA.

Verify your knowledge

- Is it possible to write queries on a subscription of a logical replication server?
Yes, it is. See the section *Exploring logical replication setup* for more details.
- Is it possible to have different fields on a subscription of a logical replication server?
Yes, it is possible to have *more* fields than we have on the publication server.
See the *Exploring logical replication setup* section for more details.
- Do I have to configure the `pg_hba.conf` file before starting logical replication?
Yes, you do. See the *Exploring logical replication setup* section for more details.
- What do I have to do if, after a DDL statement on the publication server, the subscription server does not replicate any data?
You have to replicate the DDL statement on the subscription server. See the *DDL commands* section for more details.
- Is it possible to make a logical replication starting from a physical replication?
Yes, on PostgreSQL 16 it is possible.

References

- Slony website: <https://www.slony.info>
- Logical replication: <https://www.postgresql.org/docs/current/logical-replication.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



19

Useful Tools and Extensions

This chapter is to be considered an appendix to the book. In this chapter, we will talk about some tools and extensions that allow a **Database Administrator (DBA)** to maximize the efficiency of their work by minimizing the effort needed to complete it.

We will talk about these extensions:

- `pg_trgm`
- Foreign data wrappers and the `postgres_fdw` extension

These are two of the official extensions for PostgreSQL. A site that can be very useful for finding extensions available for PostgreSQL is <https://pgxn.org/>.

In addition to extensions, we will also talk about other useful tools for the PostgreSQL DBA. There are dozens of tools available for PostgreSQL, but in this chapter, we will talk about:

- `pgbackrest`: a powerful tool useful to manage disaster recovery and **point-in-time recovery (PITR)**
- `pgloader`: a useful tool to easily migrate from MySQL, SQLite, and MS SQL to PostgreSQL; in this section, we will show an example of how to migrate from MySQL to PostgreSQL in a very easy way.

This chapter is intended to be a quick overview of some of the most useful PostgreSQL extensions and tools.

The following topics will be covered here:

- Exploring the `pg_trgm` extension
- Using foreign data wrappers and the `postgres_fdw` extension
- Managing the `pgbackrest` tool
- Exploring the `pgloader` tool

In this chapter, Docker containers are used only in some sections.

Technical requirements

This section has a Docker container, which you can find at `learn_postgresql_16/docker-images/chapter19`, so after going to the right path, let's run:

```
chapter19$ bash run-pg-docker-pg_trgm.sh chapter19-pg_trgm

postgres@learn_postgresql:~$ psql -U forum forumdb
forumdb=>
```

Exploring the `pg_trgm` extension

Now let's go back to *Chapter 13, Indexes and Performance Optimization*, in the *Indexes* section. When we talked about indexing, we learned how to make our queries faster through the use of indexes. However, B-tree indexes do not index all types of operations. Now let's consider textual data types (`char`, `varchar`, or `text`). Now, we will see that the B-tree, using the `varchar_pattern_ops` operator class, is able to index text queries for sentences that begin with `search%`, but cannot index text queries for sentences that end in `%search` or contain `%search%`:

1. Before diving into our example, let's set `enable_seqscan` to `off` in order to force PostgreSQL to use an index if it exists. We need to do this because, in our example case, PostgreSQL would always use sequential scanning by default, because we only have a few records in our table and because all data that is present in the table is stored on a single page:

```
forumdb=> set enable_seqscan to 'off';
SET
```

2. In our database, we can now execute this query on the `categories` table:

```
forumdb=> select pk,title from categories;
 pk |          title
----+-----
```

```

1 | Database
2 | Unix
3 | Programming Languages

```

- Let's create a B-tree index with the varchar opclass in order to check whether PostgreSQL uses index access to the table when we perform a query with the like operator:

```

forumdb=> create index on categories using btree(title varchar_
pattern_ops);
CREATE INDEX

```

Let's now perform some like queries:

- As our first example, let's perform a like query using a 'search%' predicate:

```

forumdb=> explain analyze select * from categories where title like
'Da%';
                                                                    QUERY PL
AN
-----
Index Scan using categories_title_idx on categories
(cost=0.13..8.15 rows=1 width=68)
(actual time=0.033..0.037 rows=1 loops=1)
  Index Cond: ((title ~>= 'Da'::text) AND (title ~< 'Db'::text))
  Filter: (title ~~ 'Da%'::text)
  Planning Time: 0.172 ms
  Execution Time: 0.075 ms
(5 rows)

```

- As the second example, let's perform a like query using a '%search' predicate:

```

forumdb=> explain analyze select * from categories where title like
'%Da%';
                                                                    QUERY PLAN
-----
Seq Scan on categories (cost=10000000000.00..10000000001.04 rows=1
width=68) (actual time=17.278..17.283 rows=1 loops=1)
  Filter: (title ~~ '%Da%'::text)
  Rows Removed by Filter: 2
  Planning Time: 0.101 ms
  JIT:

```

```

Functions: 2
Options: Inlining true, Optimization true, Expressions true,
Deforming true
Timing: Generation 0.477 ms, Inlining 5.469 ms, Optimization
7.027 ms, Emission 4.750 ms, Total 17.722 ms
Execution Time: 17.834 ms
(9 rows)

```

As we can see, only in the first case did PostgreSQL use an index approach. In the second case, PostgreSQL used a sequence scan (because there is no usable index). To improve this kind of search, we can use the `pg_trgm` extension, which is an official extension and is included in the official PostgreSQL contribs package. When we use this extension, PostgreSQL splits every word into a set of trigrams and makes a GIST or GIN index on it. For example, if we consider a word such as `dog`, its set of trigrams consists of `d`, `do`, `og`, and `dog`. Let's look at how this works in practice:

1. First of all, let's install the extension:

```

forumdb=> create extension pg_trgm;
CREATE EXTENSION

```

2. Now we can create a GIN or GIST index using the opclass `trigram`. For example, let's create a GIN index using the `gin_trgm_ops` opclass:

```

forumdb=> create index on categories using gin (title gin_trgm_
ops);
CREATE INDEX

```

3. Now let's perform our like query:

```

forumdb=> explain analyze select * from categories where title like
'Da%';
                                                                    QUERY PL
AN
-----
Scan using categories_title_idx on categories (cost=0.13..8.15
rows=1 width=68) (actual time=0.029..0.032 rows=1 loops=1)
  Index Cond: ((title ~>~ 'Da'::text) AND (title ~<~ 'Db'::text))
  Filter: (title ~~ 'Da%'::text)
Planning Time: 0.217 ms
Execution Time: 0.069 ms
(5 rows)

```

As can be seen here, PostgreSQL is now able to create an index access using a like query. The same thing happens for all types of like and ilike queries; the pg_trgm extension solves the access index for tables of this type of query. For further information about the pg_trgm extension, see <https://www.postgresql.org/docs/current/pgtrgm.html>.

The pg_trgm extension facilitates the DBA's work in all those cases where they need to optimize like and ilike queries. Now, we will move on to the next extension, postgres_fdw.

Using foreign data wrappers and the postgres_fdw extension

Foreign data wrappers allow us to access data that is hosted on an external database as if it were kept in a normal local table. We can connect PostgreSQL to various data sources, we can connect PostgreSQL to another PostgreSQL server, or we can connect PostgreSQL to another data source that can be relational or non-relational. Once the foreign data wrapper is connected, PostgreSQL is able to read the remote table as if it were local. There are foreign data wrappers for well-known databases such as Oracle and MySQL, and there are foreign data wrappers for lesser-known systems. A complete list of foreign data wrappers available for PostgreSQL is available at https://wiki.postgresql.org/wiki/Foreign_data_wrappers.

In this section, we will consider an example using the postgresql_fdw foreign data wrapper, which is used to connect a PostgreSQL server to another PostgreSQL server.

If we want to use the Docker images, we have to open two Bash terminals, and on the first one, we have to execute:

```
chapter19$ bash run-pg-docker.sh chapter19-postgresql_fdw
postgres@pg_fdw1:~$ psql -U forum forumdb
forumdb=> select * from categories;
 pk | title | description
----+-----+-----
(0 rows)
```

Then, on the second Bash terminal, we have to execute:

```
chapter19$ bash run-pg-docker-pg_fdw2.sh chapter19-postgresql_fdw
postgres@pg_fdw2:~$ psql -U forum forumdb
```

```

forumdb=> select * from categories;
 pk |          title          |          description
----+-----+-----
  1 | Database                 | Database related discussions
  2 | Unix                     | Unix and Linux discussions
  3 | Programming Languages    | All about programming languages
(3 rows)

```

Our starting situation is with two servers. We have one server called `pg_fdw1` with an IP address of `192.168.16.2` and a second server called `pg_fdw2` with an IP address of `192.168.16.3`. Our goal will be to connect server `pg_fdw2` to server `pg_fdw1` and make it possible to query the category table of server `pg_fdw2` from server `pg_fdw1` as if it were local:

1. Let's start with the installation of the `postgres_fdw` extension on the `pg_fdw1` server. So, as superuser `postgres`, let's execute:

```

postgres@pg_fdw1:~$ psql -U postgres forumdb

forumdb=# create extension postgres_fdw ;
CREATE EXTENSION

```

Suppose that on the `pg_fdw2` server, `pg_hba.conf` is configured as follows:

```

host      all             all             192.168.16.0/24
          scram-sha-256

```

2. Now we have to create the connection between the two servers, using the statement below on the `pg_fdw1` server:

```

forumdb=# CREATE SERVER remote_pg_fdw2 FOREIGN DATA WRAPPER
postgres_fdw OPTIONS(host 'pg_fdw2', dbname 'forumdb');
CREATE SERVER

```

3. Now, we have to write a user map between the two servers:

```

forumdb=# CREATE USER MAPPING FOR forum SERVER remote_pg_fdw2
OPTIONS (user 'forum', password 'LearnPostgreSQL');
CREATE USER MAPPING

```

4. Now we have to create a foreign table with the `SELECT` permission for the `forum` user:

```

forumdb=# create foreign table forum.f_categories (
          pk integer,

```

```

        title text,
        description text
    )
    SERVER remote_pg_fdw2 OPTIONS (schema_name 'forum', table_name
    'categories');
    grant SELECT ON forum.f_categories to forum;
    CREATE FOREIGN TABLE
    GRANT
    Now we can query the forum.f_categories table as if it was a local
    table:
    postgres@pg_fdw1:~$ psql -U forum forumdb
    forumdb=> select * from f_categories ;
    pk |          title          |          description
    ----+-----+-----
    1 | Database                 | Database related discussions
    2 | Unix                     | Unix and Linux discussions
    3 | Programming Languages    | All about programming languages
    (3 rows)

```

As we can see in the preceding example, we can query a foreign table as if it were on the local server.

Foreign data wrappers are very powerful tools that help with the DBA's work whenever there is a need to read data from external sources. These external sources can be represented by PostgreSQL servers, but they can also be represented by other kinds of servers: MySQL, Oracle, or SQL servers, for example.

For further information, please see <https://www.postgresql.org/docs/current/postgres-fdw.html>.

Disaster recovery with pgbackrest

In *Chapter 18, Logical Replication*, we talked about disaster recovery and PITR, and we saw how to conduct them programmatically. In the real world, a DBA has to manage multiple PostgreSQL servers and it is useful to have some tools to make life easier. The open-source world offers us a lot of solutions to address disaster recovery in an easy way. Some of these tools are listed here:

- WAL-E
- pgbarman
- OmniPITR